PlaySafe – MVP Demo

1. Introduction

This notebook is the Minimum Viable Product (MVP) of the PlaySafe project. It is a simple prototype that shows how the final models could be used by an end user, such as the technical staff of a football club.

The user gives the data of a player and the prototype returns the **probability of injury in the next season** and a risk level. The models were trained in the `FinalIteration_Model` notebook and are loaded here, so no training is needed.

The prototype has two parts:

- predicting the risk of **any player** by typing his data,
- checking the prediction for **real players** of the test seasons and comparing it with what happened.

2. Load the models and the data

The trained logistic regression and random forest are loaded from the `models` folder. The test set (the seasons 22/23 to 24/25, which the models did not see during training) is loaded from the processed data folder.

In [12]:
```
import pandas as pd
import joblib

log_reg = joblib.load("../models/log_reg.joblib")
random_forest = joblib.load("../models/random_forest.joblib")
test = pd.read_csv("../data/processed/test_set.csv")

features = ["height", "age_season", "minutes_played",
            "n_injuries", "days_out", "matches_missed", "position"]
```

3. Predict the risk of a player

The function `predict_risk` receives the data of a player in one season and returns the probability of injury in the next season given by both models. The risk level is based on the random forest, because it is the model that finds more injured players: below 40% the risk is low, between 40% and 60% it is medium and above 60% it is high. These limits are only used to make the result easier to read.

In [13]:
```
def predict_risk(height, position, age_season, minutes_played, n_injuries, days_
    player = pd.DataFrame({
        "height": [height], "position": [position], "age_season": [age_season],
        "minutes_played": [minutes_played], "n_injuries": [n_injuries],
        "days_out": [days_out], "matches_missed": [matches_missed],
```

```
    })
    prob_lr = log_reg.predict_proba(player)[0][1]
    prob_rf = random_forest.predict_proba(player)[0][1]

    if prob_rf < 0.4:
        level = "LOW"
    elif prob_rf < 0.6:
        level = "MEDIUM"
    else:
        level = "HIGH"

    print(f"Logistic regression: {prob_lr:.1%}")
    print(f"Random forest:       {prob_rf:.1%}")
    print(f"Risk level next season: {level}")

# Example: a 26-year-old midfielder with an injury history this season
predict_risk(height=180, position="Central Midfield", age_season=26,
             minutes_played=2500, n_injuries=2, days_out=45, matches_missed=7)
```

```
Logistic regression: 76.9%
Random forest:       63.5%
Risk level next season: HIGH
```

Another example, a young player with few minutes and no injuries:

In [14]:
```
predict_risk(height=180, position="Central Midfield", age_season=22,
             minutes_played=600, n_injuries=0, days_out=0, matches_missed=0)
```

```
Logistic regression: 34.0%
Random forest:       39.5%
Risk level next season: LOW
```

4. Check with real players

To see how the prototype works with real data, the function `demo_real_player` takes a player and a season from the test seasons, gives the data to the random forest and compares the prediction with what really happened in the next season. The table `examples` shows some players of the test seasons with their predicted probability and the real result.

In [15]:
```
def demo_real_player(player_name, season):
    row = test[(test["player_name"] == player_name) & (test["season"] == season)
    if row.empty:
        print("Player and season not found in the test seasons.")
        return
    prob = random_forest.predict_proba(row[features])[0][1]
    r = row.iloc[0]
    real = "was injured" if r["has_injury_next_season"] == 1 else "was NOT injur

    print(f"{player_name} ({season}): {r['age_season']} years, {r['minutes_playe
          f"{r['n_injuries']} injuries in the season")
    print(f"Predicted probability of injury next season: {prob:.1%}")
    print(f"What really happened: the player {real} in the next season")

examples = test[["player_name", "season", "has_injury_next_season"]].copy()
examples["probability"] = random_forest.predict_proba(test[features])[:, 1].roun
examples.sample(15)
```

Out[15]:

	player_name	season	has_injury_next_season	probability
51	Arsen Zakharyan	23/24	1	0.70
158	Iker MuÃ±oz	22/23	0	0.52
19	Alberto Moleiro	22/23	1	0.18
203	Juan Cruz	22/23	1	0.68
190	Jon MartÃn	22/23	0	0.04
314	Renato Tapia	22/23	1	0.25
4	Abdul Mumin	22/23	0	0.48
308	RaÃºl Albiol	22/23	1	0.50
377	Vladyslav Krapyvtsov	23/24	0	0.10
256	Max Aarons	22/23	1	0.68
128	Fernando Pacheco	22/23	0	0.46
153	Hugo Ã□lvarez	22/23	0	0.30
29	Alfonso Pedraza	22/23	1	0.80
145	Giovani Lo Celso	23/24	1	0.78
358	Thibaut Courtois	22/23	1	0.62

Now I will try looking for a specific player in the dataset in a specific season

In [16]:
```
demo_real_player("Nico Williams", "23/24")
print()
demo_real_player("Daniel Carvajal", "23/24")
```

```
Nico Williams (23/24): 21 years, 3432 minutes, 6 injuries in the season
Predicted probability of injury next season: 88.5%
What really happened: the player was injured in the next season

Daniel Carvajal (23/24): 31 years, 3957 minutes, 1 injuries in the season
Predicted probability of injury next season: 78.0%
What really happened: the player was injured in the next season
```